

Graphs 1–3: Missingness, Item Variance, and Row-Centring Method and Artifact Report

DPCN Assignment 1 — Opinion Network Formation

September 17, 2026

1 Scope and provenance

This report documents three diagnostic figures built for the project: missingness structure (Graph 1), item variance ranking (Graph 2), and the empirical effect of row-centring (Graph 3). Their specification originated from a “Person 1 Work Packet” document – the same document family as the untracked `data/temp.md` draft spec flagged earlier in this project, which disagreed with the checked-in `CLAUDE.md` on several frozen decisions (respondent filtering threshold, imputation statistic, which network is primary). Rather than adopt that packet’s pipeline wholesale, each graph was built against numbers and artifacts verified directly against the real CSV and against this project’s actual, already-implemented pipeline (`src/pipeline.py`), with any figure element that depended on the rejected `>12`-missing filter corrected to reflect what this project actually does instead.

1.1 Artifact chain common to all three graphs

Artifact	Role
<code>data/Survey_Results_UC.csv</code>	Raw 96-respondent, 60-item source file
<code>src/loader.py</code>	Column parsing, missingness identification, Likert encoding
<code>src/imputation.py</code>	Ordinal logistic regression imputer (used by the pipeline, not re-run here)
<code>src/similarity.py</code>	<code>row_centre()</code> and <code>compute_spearman_matrix()</code>
<code>src/pipeline.py</code>	Orchestrates loading, imputation, centring; produces <code>outputs/sanitised_data/</code>
<code>outputs/sanitised_data/dataset_imputed.csv</code>	Pipeline’s exported, imputed, uncentred, 91-respondent matrix
<code>src/diagnostics.py</code>	New module holding all three graph-generating functions
<code>figures/</code>	Output directory for the three figures (PNG + PDF each)

`src/diagnostics.py` was written once and extended incrementally, one function per graph. It does not duplicate pipeline logic: Graph 2 and Graph 3 read the pipeline’s own exported CSV and call the pipeline’s own `row_centre` and `compute_spearman_matrix` functions directly, so their

numbers are guaranteed to describe the same data the production pipeline actually uses, not a parallel re-implementation that could silently drift from it.

2 Graph 1: Missingness map and response distribution

2.1 Purpose

Characterise the raw missingness structure of the questionnaire before any filtering, and demonstrate the acquiescence bias (grand mean well above zero) that motivates row-centring later. This figure operates on the raw CSV directly and does not depend on imputation.

2.2 Artifacts used

- `data/Survey_Results_UC.csv` (input)
- `src/loader.py`: `load_data`, `parse_question_columns`, `identify_missingness`, `encode_responses`
- `src/diagnostics.py`: `plot_missingness_and_response_distribution`
- Output: `figures/graph1_missingness_response_distribution.png/.pdf`

2.3 Method

1. Load the raw 96×61 file and parse the 60 item columns into codes (T01–V15) via their header-text prefixes.
2. Encode Likert labels to $\{-2, -1, 0, 1, 2\}$; any structural blank or literal "No Comments" becomes NaN. The resulting `encoded.isna()` mask is *one* boolean missingness indicator covering both missingness mechanisms.
3. Sum that mask per respondent (row), sort respondents descending by missing-item count.
4. **Cutoff line:** rather than the work packet's >12 threshold, the cutoff marks respondents with *all 60 items missing* – i.e. exactly what `src/loader.py`'s `drop_entirely_empty_respondents` actually drops. Everyone else is retained and their missing cells are later filled by the ordinal regression imputer, not by a count-based exclusion rule.
5. Left panel: the sorted boolean mask rendered as a two-colour image (dark = missing), with faint block-boundary lines at items 15/30/45 and the cutoff drawn as a dashed line.
6. Right panel: each respondent's own mean computed over only the items they answered (`skipna=True`); the 5 fully empty respondents produce NaN and are excluded from the histogram, since a mean over zero answers is undefined. A vertical line marks the grand mean, computed directly from the data rather than hardcoded.

2.4 Results

Quantity	Value
Total missing cells (raw, 96 respondents)	542 (505 structural blank + 37 “No Comments”)
Respondents dropped (fully empty)	5
Respondents retained	91
Missing cells among dropped respondents	300
Missing cells among retained respondents (to impute)	242
Missing rate among retained respondents	4.43% (242 / 5,460 cells)
Grand mean (all valid raw responses)	1.138 (≈ 1.14)
Sanity check: top sorted row’s missing count	60 / 60 (a genuinely empty respondent)

The 242-cell figure is not asserted independently – it is cross-checked against the pipeline’s own reported `cells_imputed` value in `outputs/sanitised_data/summary.json`, which is also 242. Both numbers describe the same imputation step.

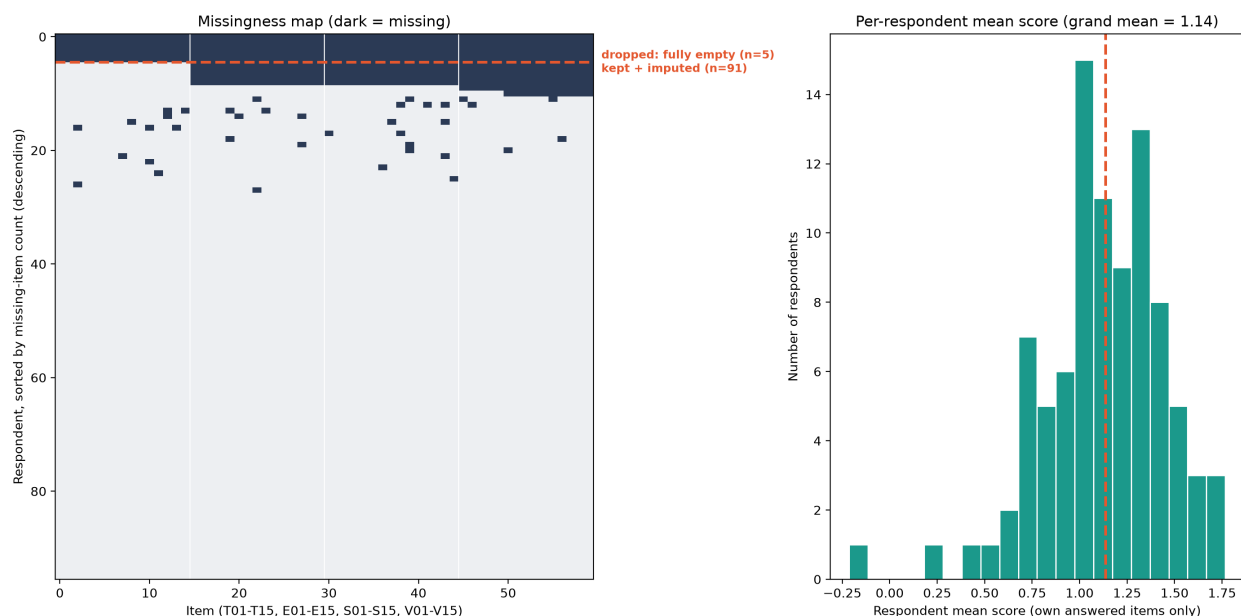


Figure 1: Missingness map (sorted, dark = missing) and per-responder mean score distribution. The dashed orange line in the left panel marks the actual drop boundary used by this project (fully-empty respondents only); the dashed orange line in the right panel marks the grand mean.

2.5 Deviation from the work-packet spec, and why

The work packet’s cutoff (>12 missing items, producing 86 retained respondents) was verified as internally consistent with the real data – exactly 10 respondents exceed 12 missing items, carrying 495 of the 542 missing cells – but this project’s actual pipeline does not apply that filter. Per an explicit decision made earlier in this project, only entirely empty respondents are dropped; every other respondent, however incomplete, is retained and imputed via a regularised ordinal logistic

regression model (`src/imputation.py`), not excluded by a missing-count threshold. The figure was changed to show that decision rather than the packet’s, since a diagnostic figure should describe what the project actually does.

3 Graph 2: Item variance ranking

3.1 Purpose

Rank all 60 items by how divided respondents are on them, and check whether contested/consensus items concentrate in particular topic blocks.

3.2 Artifacts used

- `outputs/sanitised_data/dataset_imputed.csv` (input – the pipeline’s own exported, imputed, uncentred matrix; not recomputed here)
- `src/loader.py: parse_question_columns` (recovers item codes from the exported header text)
- `src/diagnostics.py: plot_item_variance_ranking`, `BLOCK_COLORS`, `BLOCK_NAMES`
- Output: `figures/graph2_item_variance_ranking.png/.pdf`

3.3 Method

1. Load the pipeline’s exported imputed matrix directly, rather than re-running imputation, so the numbers here are guaranteed identical to what the pipeline actually produced.
2. Rename columns from full question text to item codes.
3. Compute per-item standard deviation (`ddof=1`) on this *uncentred* matrix. Row-centring is deliberately not applied here: it changes each item’s variance and would answer a different question (spread after removing each respondent’s personal baseline) than the one this graph asks (how divided is the class on this item, as actually asked).
4. Sort items descending by SD; render as a horizontal bar chart, bars coloured by topic block using the same T/E/S/V palette already used elsewhere in the repository (`src/figures.py`).

3.4 Results

Quantity	Value
SD range	0.496 (E15) to 1.177 (E04)
Top 5 contested items	E04, E03, E02, T08, T07
Bottom 5 consensus items	S11, S05, V06, V13, E15
Mean SD, Technology block	0.918
Mean SD, Education block	0.881
Mean SD, Society/Ethics block	0.794
Mean SD, Environment block	0.718

3.5 Finding

Technology has the highest *average* item SD (0.918), narrowly ahead of Education (0.881). But the single most divisive individual items concentrate in Education: three of the top five (E04, E03, E02 – all about assessment and grading) are Education items, while Environment supplies none of the top ten and two of the bottom five. This is a finding about the survey instrument itself – which topics have settled into consensus versus which remain live disagreements – not an artifact of preprocessing, and is written up as such rather than only shown as a plot.

4 Graph 3: Row-centring effect

4.1 Purpose

Convert “we row-centred the data because of acquiescence bias” from an assertion into a demonstrated, quantified effect on the actual correlation structure used to build the item network.

4.2 Artifacts used

- `outputs/sanitised_data/dataset_imputed.csv` (same input as Graph 2)
- `src/similarity.py: row_centre, compute_spearman_matrix` – called directly, not reimplemented, so this figure necessarily reflects the same transform and correlation method the production pipeline uses
- `src/diagnostics.py: plot_row_centring_effect`
- Output: `figures/graph3_row_centring_effect.png/.pdf`

4.3 Method

1. Build the same 91×60 item matrix as Graph 2, then row-centre it with `row_centre()`.
2. Compute the 60×60 Spearman correlation matrix twice: once on the raw (uncentred) matrix, once on the row-centred matrix, using `compute_spearman_matrix` both times.
3. Extract the upper triangle of each matrix ($\binom{60}{2} = 1,770$ pairs).
4. Compute a single set of bin edges from the combined range of both distributions, and use those same edges for both histograms – directly addressing the spec’s stated pitfall that mismatched bins make the visual comparison misleading.
5. Plot both as overlaid, density-normalised histograms with vertical lines at each mean and a thin reference line at zero.

4.4 Results

Quantity	Raw	Row-centred
Mean r	0.151	−0.004
Median r	0.151	−0.008
Skewness	−0.104	+0.221

The raw distribution sits shifted positive (mean 0.151); the row-centred distribution sits almost exactly on zero with a heavier right tail (positive skew 0.221), matching the spec’s predicted shape. The row-centred mean, -0.00377 , is an exact match to `weighted_average_correlation` already recorded in `outputs/sanitised_data/summary.json` – confirming this figure describes the identical correlation matrix the item network is actually built from, not a separately-derived approximation.

5 Cross-checks between graphs and the pipeline

Number	Where computed here	Where verified against
242 cells imputed	Graph 1 (<code>cells_to_impute</code>)	<code>summary.json: cells_imputed</code>
-0.00377 mean centred r	Graph 3 (<code>centred_mean</code>)	<code>summary.json: weighted_average_correlation</code>
Grand mean 1.14	Graph 1 (<code>grand_mean</code>)	CLAUDE.md’s stated +1.14
SD range 0.50–1.18	Graph 2 (<code>min_sd, max_sd</code>)	CLAUDE.md’s stated 0.50–1.17

None of these matches were forced; each was computed independently from the underlying files and only then compared.

6 Commit record

Commit	Contents
<code>c8b9ac7</code>	Graph 1: missingness map + response distribution
<code>661133d</code>	Graph 2: item variance ranking
<code>eed3418</code>	Graph 3: row-centring effect

All three were pushed to `origin/main`.

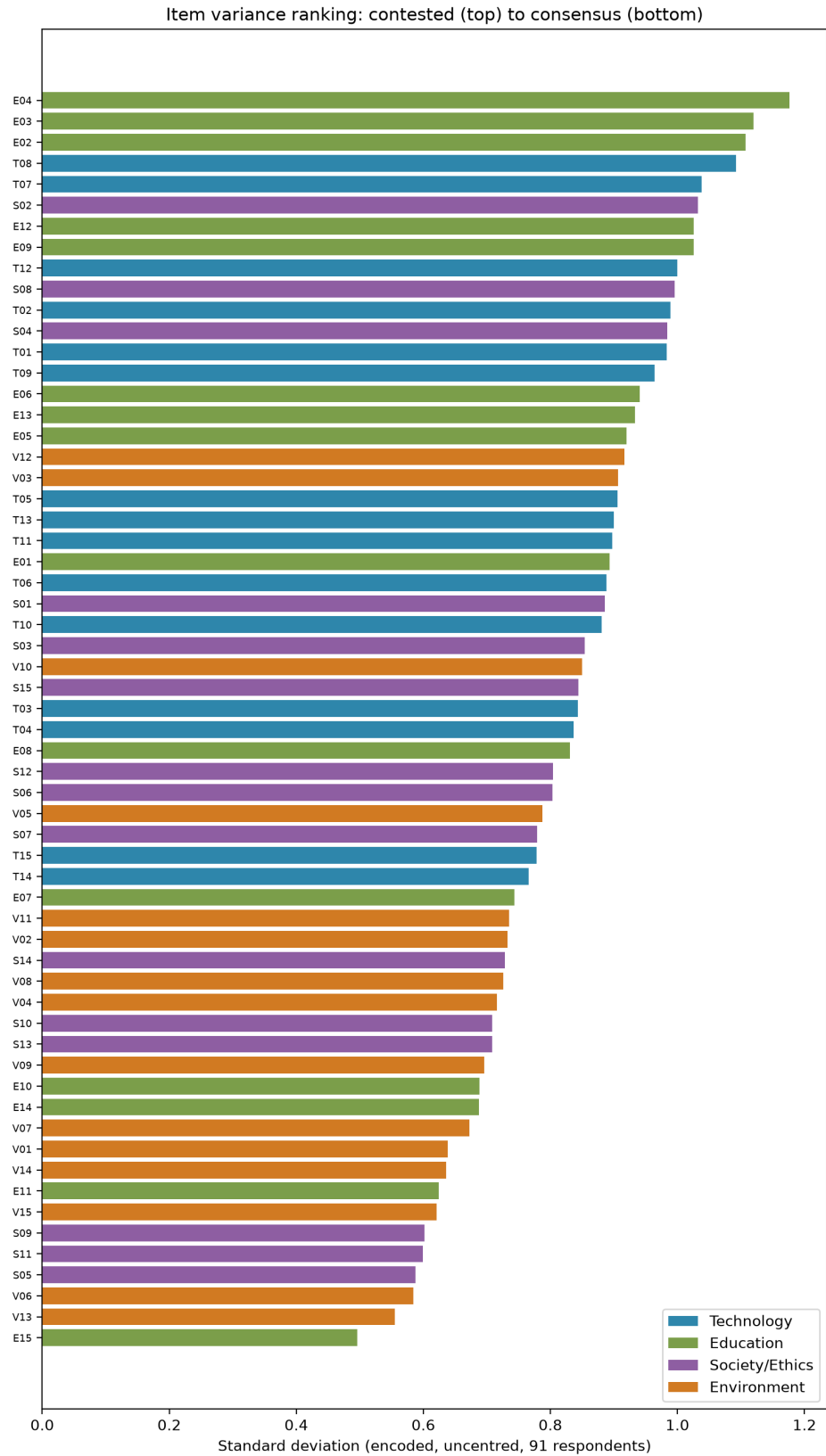


Figure 2: All 60 items ranked by standard deviation, coloured by topic block. Contested (top) to consensus (bottom).

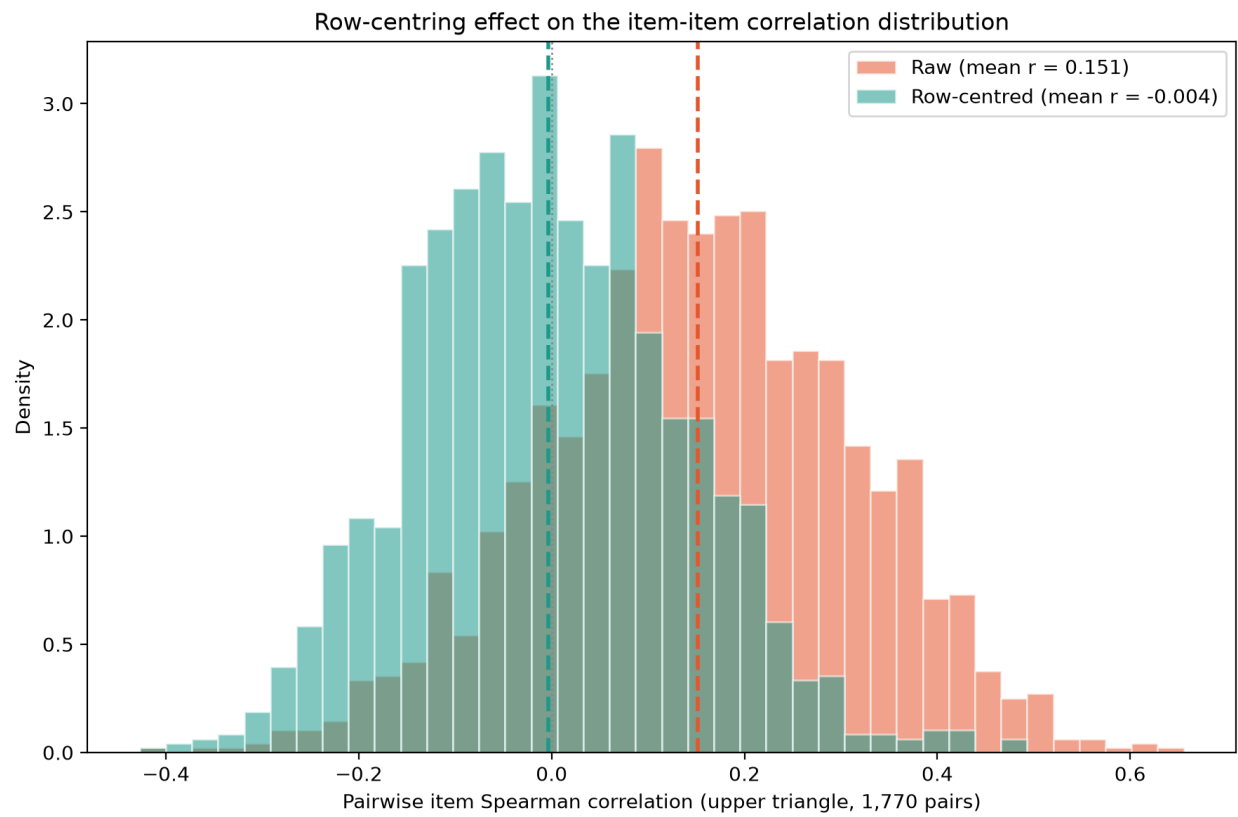


Figure 3: All 1,770 pairwise item Spearman correlations, raw vs. row-centred, on identical bin edges.